

Exploring Facebook Messenger System Design

Day 116
03 / Nov / 25

Real time Facebook messaging app, that can support millions of users.

All the chat applications like Facebook Messenger, WhatsApp, Discord etc. can be built using the same design. In this pdf we will discuss the high level architecture of the system as well as some specific features like building real-time messaging, group messaging, image and video uploads as well and push notifications.

Requirements

Functional Requirements for Facebook Messenger:

- **Real-time messaging:** instantly send and receive message
- **Groups:** which support multiple users
- **Online Status:** Show whether a user is currently online or offline
- **Media uploads:** Supporting image and video uploads
- **Read receipts:** Tick marks describing message status
- **Notifications:** Option to push notifications

Non-Functional Requirements for Facebook Messenger:

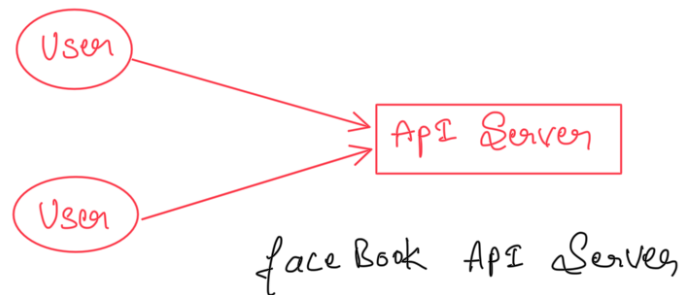
- **Low latency:** we want our system to experience very small delay times
- **High volume:** We want our system to support a high volume of requests, potentially millions of users writing messages at the same time.
- **Reliable:** Highly reliable and available all the time.
- **Secure:** Messages should be sent or received securely. Users shouldn't receive messages from random users.

Design of Facebook Messenger

Now let's discuss the overall architecture of the app. Specifically, let's see how we can send messages from one user to another:

- We have two users and we want to send a message between them.
- We need to use a chat server application, in this case, because on the internet it's actually really hard to establish a direct connection from one user to another and to make that connection reliable.
- We also want to support things like storing and retrieving chat history.
- So we need a chat server in order to establish the connection and store messages so that they can be retrieved later. A connection should be established between the two users and the

chat server.



Communication protocol for faceBook Messenger

What happens when a user A sends a message to User B?

What we require is that the user sends a message to the server and the server relays that message instantly to the user that it's intended for. However, this way breaks the model of how HTTP requests work on the internet because they can't be server-initiated rather they have to be client-initiated. So this isn't going to work and we need to come up with something else.

1. HTTP polling

Instead of just sending one request to the server, we're going to repeatedly ask the server if there is any new information available and mostly the server will reply with "no new information available". Then once in a while, it will respond, "Hey, I received a new message for you". This is probably not the right solution for this problem as we're going to send a lot of unnecessary requests to our server, which means that we're going to have a high latency so, we are only going to receive messages when we ask for them and not when they are received by the server

2. Long polling

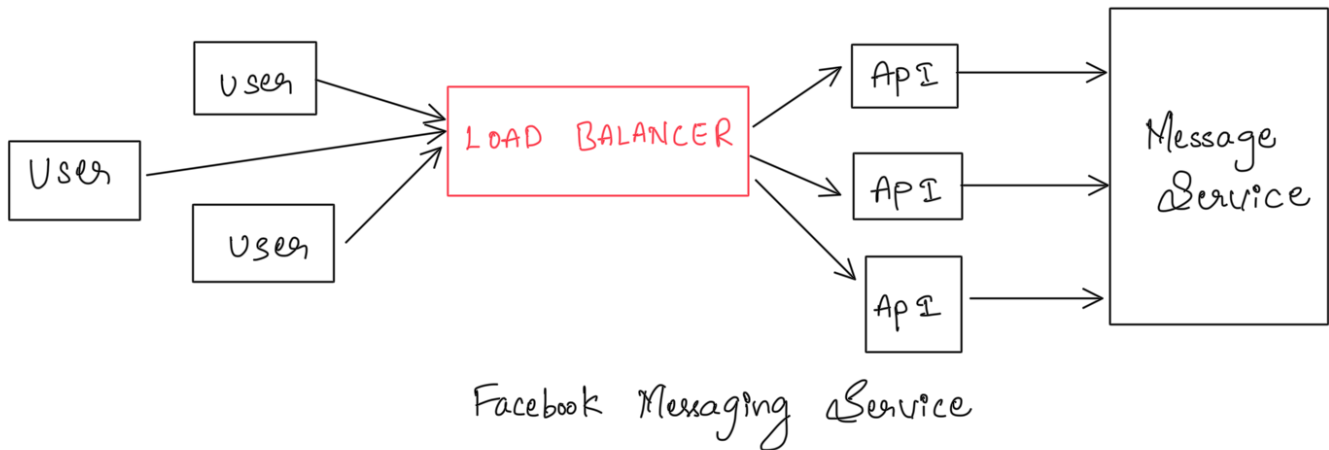
In this model, we are still using a traditional HTTP request but instead of resolving it immediately with the result, we are actually going to have the server hold on to the request and wait until data is available before it replies with the result. So, we sort of maintain an open connection with the server at all times. Once data is sent back, we immediately request a new connection and then we keep that open until the data is available. This model solves our latency problem So, while it's good for some systems like notifications but it is probably not the best for a real-time chat application.

3. Websockets

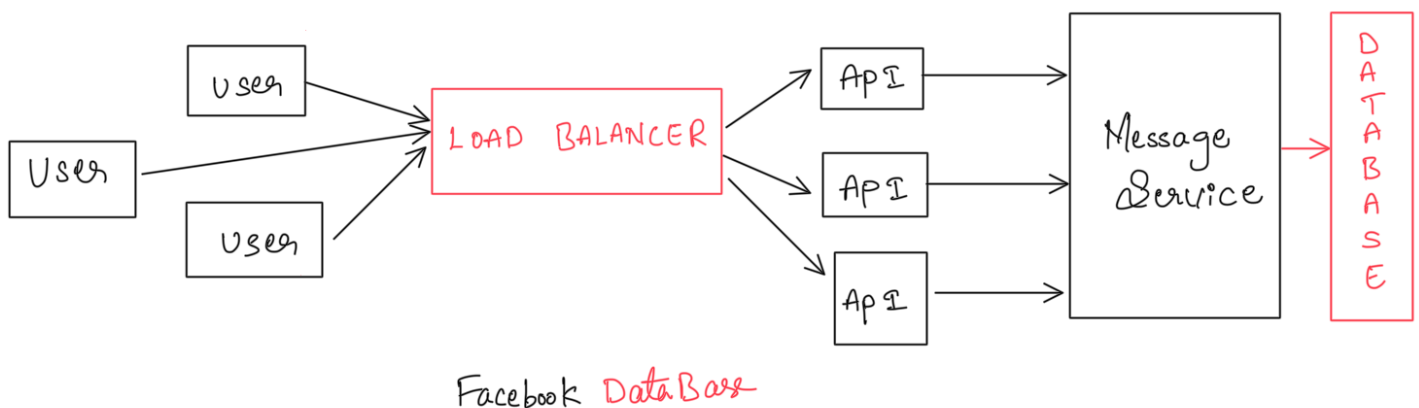
This is the solution we are going to use because it was sort of designed for this application. In websockets, we still maintain an open connection with the server but instead of just a one-way connection, it's actually a full duplex connection. So now we can send up data to the server and the server can push down data to us and this connection is maintained and kept open for the

the server can push down data to us and this connection is maintained and kept open for the duration of the session. There are some practical limitations to this solution as how many open connections a server can have at a particular time.

*Websockets is built on the TCP protocol which has about **16 bits** for the port number. This means, there's a real limitation of about **65,000** connections that any one server can have open at a time. So instead of having one API server, we are obviously going to need to have a lot of servers to handle all of these websocket connections and we're going to need a load balancer to help balance these connections. So we are going to insert a load balancer and going to draw in some API servers as shown below.*



Database Design for FaceBook Messenger



We still need to think about how we are going to store and persist these messages in our database. When we think about what kind of database to choose for application, according to our requirements we know that we want to support a really large volume of requests and store a lot of messages. We also care a lot about the availability and uptime of our service so we want to pick a database that is going to fit these requirements.

We know from things like the [CAP theorem](#) that there are going to be some sort of universal trade-offs between principles like consistency, availability, and the ability to partition or shard

our database. In our system, we want to focus on this ability to shard and partition and keep our database available rather than things like consistency which are less important in a messaging application than they would be in something like a financial application. So we would choose something like a **NoSQL** database that has built-in replication and sharding abilities so something like Cassandra or HBase would be great for this application.

Data types

So, how we are going to store and model this data in our database. We know we're going to need a few key tables and features like users, and messages and we are also going to probably need this concept of conversations, which will be groups of users who are supposed to receive messages.

- Starting with the user's table, we are going to have a unique ID for this user. We are also going to have something like a username or a name for them to display and a last active timestamp.
- We are going to need a messages table and this is going to have a unique ID but it is also going to store a reference to the user who made the message
- We are also going to have a reference or an ID for the conversation that it belongs to.
- We are also going to need the text of the message as well and if we want to support something like media uploads like images or videos we also want to store a media URL here in our database as well.
- This won't be the actual data but it will be the URL where the user can access this data to download it.

*The last thing we need is a way to query and understand which users are part of a conversation and which conversations a user is part of. So for that, we are going to add one more table which we are going to call **conversation_users** and it's just going to store the mapping from a conversation id to our user id.*

Users		Messages		Conversations		Conversations_Users	
Id	Int	Id	Int	Id	Int	Conversation	Int
Username	String	User	Int	Name	String	User	Int
Lastactive	Timestamp	Conversation	Int				
		Text	String				
		Media_Url	String				

Facebook Data types

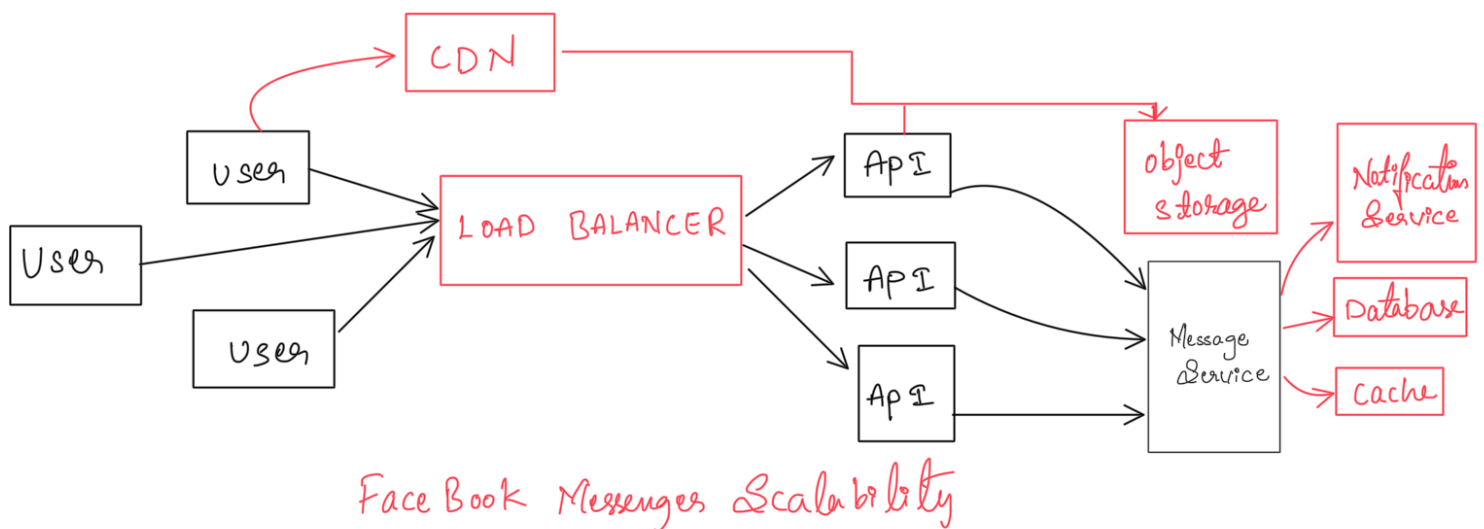
Scalability for Facebook Messenger

So in particular, one thing we are thinking about is the cost of going to our database and retrieving messages from it repeatedly so one thing we would like to add to the architecture is some sort of caching service or caching layer which would be like a read-through cache.

How we are going to store media?

Now, how we are going to store media like images and videos, and how we can upload those to the correct place. We are not going to store those in our database but instead, we're going to choose some sort of other storage platform like an object storage service like Amazon S3. Now, in order to make that more efficient we will also want to add caching. In this case, we would use something like a CDN.

The last thing we want to add in the architecture is some sort of way to notify users who are offline, about messages they may have missed. So in this case, we might want to have a notification service that is also going to be contacted by our message service in the event that the user is offline.



Conclusion

We have covered a lot of ground and we have talked about everything from choosing the right network protocol for our clients to building a distributed messaging queue system in our back end and picking the right database and data model to store all these messages. While we could go more in-depth on each of these topics hopefully we have the big picture and the overview of how we would build this application.